

Figure 3: Project and app directory structure

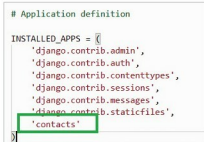


Figure 4: settings.py in the myproject directory

Open `settings.py` in the `myproject` directory and add `contacts` in the installed app (refer to Figure 4).

In `settings.py`, at the end you can see databases that show where the database and table will be created. For this example, we will use SQLite3 as the database. To create a `contacts` table, execute the following command:

```
python manage.py makemigrations
```

This will create the migrations script. You can see that the migrations directory is created in the `contacts` directory with `0001_initial.py`. Now, using the `migrate` command, create the table. It will create some other tables also.

```
python manage.py migrate
```

You can see the `db.sqlite3` file in the `myproject` directory. Our app is ready, but still has no view that will generate any output. We can use Django templates to create a view, but can omit

```
(django1.8) λ python manage.py makemigrations
Migrations for 'contacts':
  0001_initial.py:
    - Create model Contact
```

Figure 5: Output of `makemigrations` command

```
(django1.8) λ python manage.py migrate
Operations to perform:
  Synchronize unmigrated apps: staticfiles, messages
  Apply all migrations: admin, contenttypes, sessions, auth, contacts
  Synchronizing apps without migrations:
    Creating tables...
    Running deferred SQL...
    Installing custom SQL...
  Running migrations:
    Rendering model states... DONE
    Applying contenttypes.0001_initial... OK
    Applying auth.0001_initial... OK
    Applying admin.0001_initial... OK
    Applying contenttypes.0002_remove_content_type_name... OK
    Applying auth.0002_alter_permission_name_max_length... OK
    Applying auth.0003_alter_user_email_max_length... OK
    Applying auth.0004_alter_user_username_opts... OK
    Applying auth.0005_alter_user_last_login_null... OK
    Applying auth.0006_require_contenttypes_0002... OK
    Applying contacts.0001_initial... OK
    Applying sessions.0001_initial... OK
```

Figure 6: Running the `migrate` command

that section as we want a REST based interface for contacts. The next section will explore how to convert this app to a REST API.

Why REST?

By using REST we ensure that our application can be accessed from anywhere and have any front-end. Let's suppose we want to create a website using the latest technology like React or Angular, and also a mobile app or hybrid mobile app. However, if the website already exists and we want to create only a mobile app or React based app, we need to modify and return JSON instead of HTML. Instead of creating two different types of back-end logic to return HTML, JSON or some other format, we can expose our application as a REST API, which is for any client to consume and display it in the way it is capable of. Any client requesting the resource will get JSON as output, and will decide how to parse and display data.

For converting the application to a REST API, we need to use the

`rest_framework` and perform the following steps.

- Open a terminal or command prompt and execute the following command:

```
pip install django-rest-framework
```

- Open `settings.py` in the `myproject` directory, and add `rest_framework` in `INSTALLED_APPS`.
- Create new files named `serializers.py`, `views.py` and `urls.py` in the `contacts` directory.
For any model to be exposed as REST we need to create a `serializer` class in `serializer.py`. We will create the `ContactSerializer` class with all the fields. Open `serializer.py` and add the following code to it:

```
from rest_framework import serializers
from models import Contact
```

```
class ContactSerializer(serializers.
ModelSerializer):
    class Meta:
        model = Contact
        fields = '__all__'
```